

Compte rendu script

Table des matières

Introduction	2
Analyse et Recherche	2
Le cours théorique du matin :	2
Décryptage du script d'exemple (learn.sh) :	2
• Le menu persistant (Bloc H) :	2
• La gestion des choix (Bloc D) :	2
• L'expérience utilisateur (Bloc 1) :	2
Mes choix techniques (et contraintes de temps)	2
Planification de la journée	3
Conclusion	3

Introduction

Pour ce TP d'administration système (SISR), l'idée de départ était claire : créer un script Bash qui serve de véritable boîte à outils pour nos commandes Linux. Dans ce compte rendu, **on** va retracer toute la conception de cet utilitaire, de l'analyse des ressources fournies jusqu'au script final 100 % fonctionnel. Surtout, **on** va s'attarder sur le principe fondamental qui se cache derrière ce travail : comprendre comment automatiser une suite d'instructions.

Analyse et Recherche

Avant de taper la moindre ligne de code, **on** a d'abord fait le point sur les ressources fournies, histoire de partir sur de bonnes bases.

Le cours théorique du matin : Le cours de 11h à midi a vraiment posé les bases.

C'est là que j'ai saisi l'architecture classique d'un script Bash : le fameux Shebang (`#!/bin/bash`) pour appeler le bon interpréteur, la façon de déclarer des variables, et surtout l'utilité des structures de contrôle pour automatiser nos tâches d'administration.

Décryptage du script d'exemple (learn.sh) : En début d'après-midi, j'ai épluché le script fourni par le prof. L'idée, c'était de repérer les morceaux de code que je pouvais recycler pour mon propre outil. Je me suis concentré sur trois mécanismes clés :

- **Le menu persistant (Bloc H) :** J'ai regardé comment la boucle `while true` maintient le script ouvert en continu. C'est le détail qui change tout et transforme un simple fichier de commandes en un vrai programme interactif.
- **La gestion des choix (Bloc D) :** En étudiant la structure `case`, j'ai vite compris que c'était la façon la plus propre et lisible de gérer les saisies de l'utilisateur (1, 2 ou 'q' pour quitter), bien plus efficace qu'une interminable série de `if/else`.
- **L'expérience utilisateur (Bloc 1) :** J'ai repéré une fonction de pause très pratique basée sur `read -r -n 1 -s`. Je l'ai tout de suite mise de côté pour l'intégrer à mon projet. Ça laisse le temps de lire l'écran avant qu'il ne s'efface, ce qui est quand même beaucoup plus confortable à l'usage.

Mes choix techniques (et contraintes de temps) : Pendant mes recherches, je me suis aussi penché sur la lecture de fichiers externes (les TXT ou CSV avec `while IFS= read`). J'ai bien compris la logique. Par contre, sachant que je devais impérativement partir à 15h pour un rendez-vous médical, j'ai dû trancher. Plutôt que de me lancer dans une fonctionnalité complexe au risque de rendre un code buggé ou inachevé, j'ai préféré jouer la carte de la sécurité : livrer un script plus simple, mais 100 % robuste et fonctionnel.

Planification de la journée

Pour mener à bien ce projet dans le temps imparti, j'ai dû structurer ma journée de manière assez précise. Voici comment je me suis organisé :

- **11h00 - 12h00** : Cours d'initiation aux scripts avec le formateur. C'était l'occasion de découvrir (et de comprendre) les concepts de base : le fameux *shebang*, le fonctionnement des variables, les boucles `while` et les conditions.
- **13h30 - 14h00** : De retour de pause, j'ai pris une demi-heure pour analyser le script d'exemple `learn.sh` fourni par le prof. L'objectif était de bien capter la logique qui se cache derrière la création d'un menu interactif.
- **14h00 - 14h30** : Passage à la pratique avec le développement de mon propre script d'aide-mémoire. Avec le chronomètre qui tournait, j'ai assumé le choix de me concentrer sur une seule version, en privilégiant un code propre et qualitatif plutôt que de multiplier les fonctionnalités.
- **14h30 - 14h50** : Phase de validation. J'ai effectué mes tests de fonctionnement, pris les captures d'écran nécessaires et rédigé ce compte rendu directement sur le template Word.
- **15h00** : Fin du TP pour moi. J'ai déposé mon travail finalisé sur la plateforme juste avant mon départ anticipé pour mon rendez-vous médical.

Conclusion

Au final, ce TP m'a surtout permis de bien ancrer ce qu'est réellement un script. Derrière le jargon technique, c'est tout simplement un fichier texte que l'interpréteur (Bash, dans notre cas) lit ligne par ligne pour exécuter une série de commandes. En me mettant dans la peau d'un administrateur réseau, j'ai vite compris que c'était l'outil indispensable : ça permet d'automatiser les corvées répétitives, de limiter les erreurs de frappe, et surtout, de gagner un temps fou au quotidien.